EXVUL

SMART CONTRACT AUDIT REPORT

Hedgehood Vaults Smart Contract

OCTOBER 2026

Contents

1. EXECUTIVE SUMMARY **4**
1.1 Methodology 4

2. FINDINGS OVERVIEW **8**
2.1 Project Info And Contract Address 8
2.2 Summary 8
2.3 Key Findings 9

3. DETAILED DESCRIPTION OF FINDINGS **10**
3.1 Stale price fallback miscounts market closures as outage time 10
3.2 Settlement callers can choose conflicting late-fallback outcomes 12
3.3 PortfolioRouter can silently skip a due claim at cutoff 14
3.4 The rotator can short-pay its liquidity provider 15
3.5 Permissionless creation can make full basket discovery unusable 16

4. CONCLUSION **18**

5. APPENDIX **19**
5.1 Basic Coding Assessment 19
5.1.1 Apply Verification Control 19
5.1.2 Authorization Access Control 19
5.1.3 Forged Transfer Vulnerability 19
5.1.4 Transaction Rollback Attack 20
5.1.5 Transaction Block Stuffing Attack 20
5.1.6 Soft Fail Attack Assessment 20
5.1.7 Hard Fail Attack Assessment 21
5.1.8 Abnormal Memo Assessment 21
5.1.9 Abnormal Resource Consumption 21
5.1.10 Random Number Security 22
5.2 Advanced Code Scrutiny 22
5.2.1 Cryptography Security 22
5.2.2 Account Permission Control 22
5.2.3 Malicious Code Behavior 23
5.2.4 Sensitive Information Disclosure 23
5.2.5 System API 23

6. DISCLAIMER **24**

7. REFERENCES **25**

8. About Exvul Security **26**

1. EXECUTIVE SUMMARY

ExVul Web3 Security audited the frozen Hedgehood Vaults smart contract baseline and reviewed the patched source delivery dated 28 October 2026. The exact baseline commit is recorded in Section 2.1. The audit records one High, one Medium, and three Low findings. The patched source addresses all five findings, which are marked **FIXED** in this report.

The audited scope covers OptionVault v2 and the share-collateral module used for Morpho Blue collateralization. The collateral review includes `ShareRiskOracle` fallback valuation: collateral-01 is detailed in Section 3.1 and is FIXED by counting only open-market outage time while preserving accrued decay through closures. The disputed collateral-02 candidate was re-evaluated, found not applicable to the configured RHNVDA feed behavior, and remained unchanged.

The review compared the vulnerable and patched paths, checked manifest hashes, and exercised focused Foundry regressions. The non-fork v2 suite passed 81 of 81 tests. For the share-collateral code, the focused collateral-01 set passed 5 of 5 tests and the related `ReviewFixes` set passed 6 of 6 tests. A broader `ShareRiskOracle` file run under locally installed Foundry 1.3.5 passed 20 of 22 tests; the two failures are unrelated pre-existing constructor-revert harness incompatibilities against the scope-pinned Foundry 1.8.4 toolchain.

Two disputed candidates, v2-01 and collateral-02, were re-evaluated and excluded from the findings after their disputes were accepted. v2-01 is consistent with the protocol's raw-token unit model, and collateral-02 is not applicable to the configured RHNVDA feed behavior observed for the deployment.

The fixes are not yet deployed. The immutable live v2 contracts retain the pre-fix behavior; this report's FIXED status applies to the reviewed source delivery, not to production deployment.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

IMPACT

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Hedgehood Vaults	21/10/2026 - 29/10/2026	Solidity

Repository
https://github.com/keyuyuan/rh

Commit Hash
698276a627972a50822d18b21239203a6ec286e2

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	1
MEDIUM	1
LOW	3
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Stale price fallback miscounts market closures as outage time	Fixed
MEDIUM	Settlement callers can choose conflicting late-fallback outcomes	Fixed
LOW	PortfolioRouter can silently skip a due claim at cutoff	Fixed
LOW	The rotator can short-pay its liquidity provider	Fixed
LOW	Permissionless creation can make full basket discovery unusable	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Stale price fallback miscounts market closures as outage time

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/src/collateral/ShareRiskOracle.sol`

DESCRIPTION:

The frozen implementation held the latest feed print whenever the calendar reported a scheduled closure, then used wall-clock age after reopening. Consequently, a weekend or holiday was retroactively counted as outage time at reopen. Conversely, entering a scheduled closure after an unexpected outage returned the full latest print and erased decay that had already accrued.

```
if (_marketClosed()) return (last, false);
uint256 age = block.timestamp - updatedAt;
if (age <= holdAge) return (last, false);
uint256 cut = ((age - holdAge) / 1 hours) * decayBpsPerHour;
```

The source mechanism was reachable whenever the primary stock oracle failed and the fallback feed was used around a closure boundary. No historical exploitation was established.

IMPACT:

At reopen, the fallback could decay earlier than intended even though no trading session had elapsed. During a later closure, already-accrued decay could be undone. Because this value feeds share-collateral NAV, either direction could distort borrowing health and liquidation decisions.

RECOMMENDATIONS:

Count only the portions of calendar sessions during which the market was open. Detect boundary support once at construction, walk `nextOpen` and `nextClose`, cap the walk at the decay horizon, and preserve accrued decay across closures. If the calendar lacks usable boundaries, retain the conservative wall-clock fallback.

```
if (countsOpenTime) {
    uint256 horizon =
        decayBpsPerHour == 0 ? holdAge : holdAge + Math.ceilDiv(BPS -
    floorBps, decayBpsPerHour) * 1 hours;
    age = _openAge(updatedAt, horizon, closed);
} else {
    if (closed) return (last, false);
    age = block.timestamp - updatedAt;
}
```

The patched implementation follows this design. Focused regressions verify weekend and holiday reopening, an outage spanning a weekend, and a closure that preserves prior decay.

3.2 Settlement callers can choose conflicting late-fallback outcomes

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/src/v2/OptionBook.sol`

`contracts/src/v2/EpochFund.sol`

DESCRIPTION:

For an unpriced book after `FORCE_SETTLE_DELAY`, the frozen implementation left two permissionless settlement paths open. A direct `OptionBook.settle` treated the zero-price fallback as a normal, non-forced settlement and charged book fees. `EpochFund.forceSettle` instead used the forced zero outcome, returned the full premium to NAV, and zeroed the engine caps. Transaction ordering therefore selected the economic result.

```
if (price == 0 && epochPremium != 0 && block.timestamp < uint256(expiry)
    + FALLBACK_DELAY) revert TooEarly();
bool chargeFees = !forced || price != 0;
```

IMPACT:

After the force deadline, any caller could choose whether an unpriced book paid fees or returned the entire premium to fund NAV. This produced inconsistent depositor and fee-recipient accounting for the same epoch state and created a race between settlement callers.

RECOMMENDATIONS:

From `FORCE_SETTLE_DELAY`, reject direct unpriced settlement and require the fund's bounded force-settlement path. Continue to permit direct settlement after the deadline when a finalized nonzero price exists.

```
if (!forced && price == 0 && block.timestamp >= uint256(expiry) +
    fund.FORCE_SETTLE_DELAY()) {
     revert ForceSettlementRequired();
}
```

```
bool chargeFees = !forced || price != 0;
```

The guard is present in the patched `OptionBook`. Regression tests confirm that an unpriced book has only the fund-controlled outcome from the force deadline, while a priced book still settles normally.

3.3 PortfolioRouter can silently skip a due claim at cutoff

SEVERITY: LOW **STATUS:** Fixed

PATH:

`contracts/src/v2/PortfolioRouter.sol`

DESCRIPTION:

The frozen router previewed an account before asking the fund to freeze and credit a cohort that had just reached its cutoff. If the due cohort was not yet materialized in the claim ledger, `previewAccount` returned zero and the router skipped `claim`, even though the fund's claim path would have credited the account first.

```
(, uint256 s, uint256 u) =
    IEpochFund(vaults[i]).previewAccount(msg.sender);
if (s != 0 || u != 0) IEpochFund(vaults[i]).claim(msg.sender, msg.sender);
```

IMPACT:

A portfolio claim submitted at the cutoff could report success while leaving a due claim unprocessed. Funds were not lost, but the user had to discover the omission and submit another transaction after the cohort was credited.

RECOMMENDATIONS:

Credit each account before previewing its claimable balances. Preserve the zero-balance check to avoid deliberately reverting `claim` when nothing is due.

```
IEpochFund(vaults[i]).creditAccount(msg.sender);
(, uint256 s, uint256 u) =
    IEpochFund(vaults[i]).previewAccount(msg.sender);
if (s != 0 || u != 0) IEpochFund(vaults[i]).claim(msg.sender, msg.sender);
```

The patched ordering is correct, and the cutoff regression confirms that the router freezes, credits, and claims the due cohort in one transaction.

3.4 The rotator can short-pay its liquidity provider

SEVERITY: LOW **STATUS:** Fixed

PATH:

`contracts/src/v2/RfqRotator.sol`

DESCRIPTION:

The frozen callback used `safeTransfer` to send the signed `amountOut` to the LP but did not verify the LP's received balance. A fee-on-transfer or otherwise non-standard output token could debit the rotator by the requested amount while crediting the LP less, after which the LP's full `amountIn` was still pulled into the vault.

```
outToken.safeTransfer(_activeLp, amountOut);
inToken.safeTransferFrom(_activeLp, msg.sender, _activeAmountIn);
```

IMPACT:

The LP could receive less than the amount it signed while delivering the full counter-asset. The discrepancy was bounded by the individual quote but violated its exact exchange terms.

RECOMMENDATIONS:

Measure the LP's output-token balance immediately before and after the transfer and revert unless it increases by exactly `amountOut`. The revert must occur before pulling the LP's input asset.

```
uint256 lpBefore = outToken.balanceOf(_activeLp);
outToken.safeTransfer(_activeLp, amountOut);
if (outToken.balanceOf(_activeLp) != lpBefore + amountOut) revert
   UnsafeAssets();
inToken.safeTransferFrom(_activeLp, msg.sender, _activeAmountIn);
```

The patched callback implements the exact-receipt invariant. A regression with a taxed output token confirms that the rotation reverts atomically.

3.5 Permissionless creation can make full basket discovery unusable

SEVERITY: LOW **STATUS:** Fixed

PATH:

`contracts/src/v2/BasketFactory.sol`

DESCRIPTION:

Basket creation is permissionless, while the frozen discovery function returned the entire storage array in one call. Any account could create enough baskets that `baskets()` became too expensive for an on-chain caller or exceeded practical RPC response and gas limits.

```
function baskets() external view returns (address[] memory) {
    return _baskets;
}
```

IMPACT:

Full-list discovery could become unusable as the registry grew. Existing baskets and creation remained functional, but integrations depending on the unbounded getter could fail or require prohibitively large calls.

RECOMMENDATIONS:

Expose constant-size indexed discovery and rely on `BasketCreated` events for bulk indexing. Clients can query the count and fetch entries in bounded batches.

```
function basketCount() external view returns (uint256) {
    return _baskets.length;
}

function basketAt(uint256 index) external view returns (address) {
    return _baskets[index];
}
```

The patched factory removes the unbounded getter and exposes `basketCount()` and `basketAt()`. Tests confirm count and indexed lookup after permissionless creation.

4. CONCLUSION

In this audit, we analyzed the Hedgehood Vaults OptionVault v2 and share-collateral smart contract implementation. The five findings are described in Section 3, and each patched source path addresses the reported mechanism with focused regression coverage. We therefore consider the reviewed smart contract audit result to be **PASSED**.

The FIXED statuses describe the reviewed source delivery. The fixes were not deployed at the time of verification, and the immutable live contracts retain their earlier behavior. Deployment should be independently verified before relying on the FIXED status for production risk.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul